

BTS SERVICES INFORMATIQUES AUX ORGANISATIONS	SESSION 2026
Épreuve E5 - Conception et développement d'applications (option SLAM)	
ANNEXE 7-1-B : Fiche descriptive de réalisation professionnelle (recto)	

DESCRIPTION D'UNE RÉALISATION PROFESSIONNELLE		N° réalisation : 6
Nom, prénom : Beaulieu-Luangkham Axel		N° candidat :02542565481
Épreuve ponctuelle	Contrôle en cours de formation ☒	Date :/..../....
Organisation support de la réalisation professionnelle Groupe Technicolor		
Intitulé de la réalisation professionnelle Création d'une toolbox dans nuke, maya et houdini sur la base d'une toolbox créée par un artiste VFX		
Période de réalisation : 01/09/2025 à 01/03/2026 Lieu : Site Bastille 11 boulevard de la bastille		
Modalité : Seul(e) En équipe ☒		
Compétences travaillées ☒ Concevoir et développer une solution applicative ☒ Assurer la maintenance corrective ou évolutive d'une solution applicative ☒ Gérer les données		
Conditions de réalisation⁵ (ressources fournies, résultats attendus) Utilisation du code original d'une toolbox dans maya nuke et houdini écrit par un artiste VFX qui n'était pas déployée proprement dans le pipeline de l'entreprise. Ticket sur notion comme cahier des charges des étapes de réalisation de cette toolbox Résultats attendus : - Création d'une toolbox dans nuke maya et houdini permettant aux artistes d'accéder à leur propres scripts, d'ouvrir sur leur machine le code d'un script, de créer un nouveau script, d'ouvrir un asset/scène houdini ou nuke et de sauvegarder une scène dans houdini et nuke. - Utilisation dans un premier temps du wrapper Qt python pour créer l'UI de la toolbox - Rendre l'application durable contrairement à la toolbox d'origine pour simplifier grandement l'ajout futur de fonctionnalités diverses - Déploiement généralisé de la toolbox dans le pipeline sur maya nuke et houdini		
Description des ressources documentaires, matérielles et logicielles utilisées⁶ Ticket Notion, aiguillage par l'artiste VFX à l'origine de la première toolbox, travail en collaboration avec 2 collègues pour le développement de l'UI et de tests notamment. Langage : python, utilisation du wrapper Qt, de l'api maya nuke et houdini logiciels : Visual studio code, notion, vim, nuke, maya, houdini		
Modalités d'accès aux productions⁷ et à leur documentation⁸ Le code de la toolbox est disponible uniquement sur le gitlab de l'entreprise et certaines machines en local.		

⁵ En référence aux *conditions de réalisation et ressources nécessaires* du bloc « Conception et développement d'applications » prévues dans le référentiel de certification du BTS SIO.

⁶ Les réalisations professionnelles sont élaborées dans un environnement technologique conforme à l'annexe II.E du référentiel du BTS SIO.

⁷

⁸

Épreuve E5 - Conception et développement d'applications (option SLAM)

ANNEXE 7-1-B : Fiche descriptive de réalisation professionnelle
(verso, éventuellement pages suivantes)

Descriptif de la réalisation professionnelle, y compris les productions réalisées et schémas explicatifs

- Réécriture de tout le code logique de la toolbox sous une architecture pseudo MVC (pas d'accès à la base de données ici) avec une partie UI, un contrôleur qui reçoit les actions faites sur l'UI et les traite pour envoyer des données à traiter aux actionneurs voulus dans le core, et la partie core qui exécute les actions demandées par le contrôleur.
- Respect des fonctionnalités initialement demandées
- Création d'une UI à l'aide du wrapper Qt python en premier lieu, puis après une phase de débogage, l'UI ne se chargeant pas correctement dans certains logiciels, mise à jour de celle-ci en utilisant l'api native de Qt plutôt que le wrapper maison
- Ecriture de quelques tests pour assurer le lancement de l'UI avec les bons dictionnaires de scripts selon l'environnement de prod
- Déploiement de la toolbox dans toutes les prods sur nuke maya et houdini

```
13 import os
14
15 from vfxDccScripts.toolbox.api import (
16     ScriptTreeBuilder,
17     LaunchManager,
18     ToolBoxMainWindowUi,
19 )
20 from vfxDccScripts.toolbox.toolbox_main import ToolboxMain
21
22 You, 23 hours ago | 1 author (You)
23 class Launcher():
24
25     _TOOLBOX_ENV_VAR = ["TOOLBOX_STUDIO_SCRIPTS", "TOOLBOX_USER_SCRIPTS", "TOOLBOX_PROD_SCRIPTS"]
26
27     def __init__(self):
28         self._tree_builder = ScriptTreeBuilder()
29         self._ui_service = ToolboxMain()
30         self._session_checker = LaunchManager
31
32     def launch(self):
33         current_dcc = self._session_checker.get_launcher()
34
35         for env_var in self._TOOLBOX_ENV_VAR:
36             if env_var in os.environ:
37                 self._tree_builder.register_folder(os.path.join(os.environ[env_var], "common"))
38                 self._tree_builder.register_folder(os.path.join(os.environ[env_var], current_dcc))
39
40         tree = self._tree_builder.create_scripts_tree_from_path()
41
42         toolbox_main_ui = ToolBoxMainWindowUi(self._ui_service, tree)
43         toolbox_main_ui.show()
44
```